

Managing Amazon Redshift Access

SPL-BE-200-ANDEW2-1 - Version 1.0.2

Lab overview

Now that you have finished performance tuning your data warehouse, AnyCompany Events wants you to explore Amazon Redshift security settings and features. To do this, you encrypt the Amazon Redshift Serverless endpoint and create several new Amazon Redshift roles to make sure employees have the right level of access using column-level and row-level security. Then, using dynamic data masking, you protect sensitive data in the data warehouse with a masking policy.

AnyCompany Events also wants you to set up database audit logging with Amazon CloudWatch and Amazon Simple Storage Service (Amazon S3) so the database administrators can view connections and user activities in the TICKIT database. The database administrator uses these logs for security and troubleshooting purposes when they perform database audits.

In this lab, you secure your Amazon Redshift resources. You set up Redshift endpoint encryption, create and manage users and roles, apply column-level security, apply row-level security, configure dynamic data masking, and review audit logs in Amazon CloudWatch and Amazon S3.

Objectives

By the end of this lab, you should be able to do the following:

- Create and manage users and roles.
- Apply and manage column-level security.
- Apply and manage row-level security.
- Configure dynamic data masking.
- Review audit logs.

Technical knowledge prerequisites

This lab requires the following prerequisites:

- Access to a computer with Microsoft Windows, Mac OS X, or Linux (Ubuntu, SuSE, or Red Hat)
- A modern internet browser such as Chrome or Firefox
- Basic familiarity with cloud and AWS services

Icon key

Various icons are used throughout this lab to call attention to different types of instructions and notes. The following list explains the purpose for each icon:

- **Note:** A hint, tip, or important guidance.
- **Expected output:** A sample output that you can use to verify the output of a command or edited file.
- **Command:** A command that you must run.

- **Consider:** A moment to pause to consider how you might apply a concept in your own environment or to initiate a conversation about the topic at hand.
- **Task complete:** A conclusion or summary point in the lab.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.

Caution: You must wait for the provisioned AWS services to be ready before you can continue.

2. To open the lab, choose **Open Console**.

You are automatically signed in to the AWS Management Console in a new web browser tab.

Warning: Do not change the **Region** unless instructed.

Common sign-in errors

Error: Choosing Start Lab has no effect

In some cases, certain pop-up or script blocker web browser extensions might prevent the **Start Lab** button from working as intended. If you experience an issue starting the lab:

- Add the lab domain name to your pop-up or script blocker's allow list or turn it off.
- Refresh the page and try again.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Task 1: Enable audit logging for Amazon Redshift Serverless

Audit Logging is good for troubleshooting, monitoring, and security purposes, making it possible to determine suspicious queries by checking the connections and user logs to see who is connecting to the database.

In this task, you configure Amazon Redshift Serverless to export connection, user, and user-activity log data to a log group in Amazon CloudWatch Logs.

Amazon Redshift logs information in the following log files:

- **Connection log:** Logs authentication attempts, connections, and disconnections.
- **User log:** Logs information about changes to database user definitions.

- **User-activity log:** Logs each query before it's run on the database.
- At the top of the AWS Management Console, in the search bar, search for and choose **Amazon Redshift**.
 - On the Serverless dashboard, choose the **labnamespace** link.
 - Choose the **Security and encryption** tab.
 - In the **Security and encryption** section, choose **Edit**.
 - Check all the boxes to select the log types under **Export these logs**.
 - Choose **Save changes**.

Audit logging is now turned on.

Task complete: You have successfully configured audit logging in Amazon Redshift Serverless.

Task 2: Create and manage users and roles

In this task, you create and manage database users and roles in Amazon Redshift. Proper configuring of users and permissions is crucial to control access, enforce security, and manage governance of data.

- Copy the **CommandHostUrlRedShift** value found in the left pane of these instructions into a new browser tab to access the command host terminal.
- Command:** To connect to Redshift endpoint using psql, run the following command on the command host:
 - `cd ~`
 - `echo $RedshiftEndpoint`
 - `echo $RedshiftDB`

```
psql -U dbadmin -h $RedshiftEndpoint -d $RedshiftDB -p 5439
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****

psql (15.6, server 8.0.2)
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-
SHA384, bits: 256, compression: off)
Type "help" for help.

ticketdb=#
```

Note: It is recommended to create users with access tailored to job function and match the access level to job roles to improve security.

Create read-only users and roles

11. **Command:** To create a read-only role in the sales schema, run the following command:

```
create role sales_ro;
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
CREATE ROLE
```

12. **Command:** To create a new database user (data analyst) named *Jane Doe*, run the following command:

13. `create user jane_doe with password 'AwsRSuser@123'`
14. `valid until '9999-01-01'`
15. `session timeout 3600`
`connection limit 2;`

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
CREATE USER
```

13. **Command:** To grant the schema usage and select access to the objects in the sales schema, run the following command:

14. `grant usage on schema sales to role sales_ro;`
`grant select on all tables in schema sales to role sales_ro;`

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
GRANT  
GRANT
```

14. **Command:** To assign the role sales_ro to jane_doe, run the following command:

```
grant role sales_ro to jane_doe;
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

GRANT

To validate if read-only access is working, you now login as john_doe and attempt to run some queries with write operations.

15. **Command:** To connect to the Redshift endpoint as jane_doe user, run the following commands on the command host:

```
quit
```

```
psql -U jane_doe -h $RedshiftEndpoint -d $RedshiftDB -p 5439 -W
```

16. When prompted for password for the jane_doe user, enter

```
AwsRSuser@123
```

.

Expected output:

```
*****  
****  EXAMPLE OUTPUT  ****  
*****
```

```
psql (15.6, server 8.0.2)  
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-  
SHA384, bits: 256, compression: off)  
Type "help" for help.
```

```
ticketdb=#
```

17. **Command:** To verify that you have logged in as the jane_doe user, run the following command:

```
select current_user;
```

Expected output:

```
*****  
****  EXAMPLE OUTPUT  ****  
*****
```

```
current_user  
-----  
jane_doe  
(1 row)
```

18. **Command:** To validate the read-only permissions for the jane_doe user, run the following analytics query that finds the top five sellers in San Diego by total tickets sold in 2008:

19. `SET search_path TO sales;`

20.

21. `select sellerid, username, (firstname || ' ' || lastname) as name,`

22. `city, sum(qtysold)`

23. `from sales, date, users`

```

24. where sales.sellerid = users.userid
25. and sales.dateid = date.dateid
26. and year = 2008
27. and city = 'San Diego'
28. group by sellerid, username, name, city
29. order by 5 desc
    limit 5;

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

SET
sellerid | username |      name      | city | sum
-----+-----+-----+-----+-----
    49977 | JJK84WTE | Julie Hanson   | San Diego | 22
    19750 | AAS23BDR | Charity Zimmerman | San Diego | 21
    29069 | SVL81MEQ | Axel Grant     | San Diego | 17
    43632 | VAG08HKW | Griffin Dodson  | San Diego | 16
    36712 | RXT40MKU | Hiram Turner   | San Diego | 14
(5 rows)

```

19. **Command:** To validate the read-only restriction for user *jane_doe*, run the following INSERT statement:

```

INSERT into event values (8999,128,9,2049,'Terry Whitlock','2010-
10-11 20:00:00');

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

ERROR:  permission denied for relation event

```

Note: Since the Data Analyst user *jane_doe* has read-only access configured, attempting to run this INSERT statement failed with an access error. This verifies the read-only restriction blocks write operations as expected.

20. **Command:** To validate that *jane_doe* is assigned to the read-only *sales_ro* group, run the following command:

```

select user_is_member_of('jane_doe', 'sales_ro');

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

user_is_member_of
-----
t

```

(1 row)

Task complete: You have successfully created and configured users with specific access permissions by leveraging roles in an Amazon Redshift Serverless endpoint.

Task 3: Apply and manage column-level security

Amazon Redshift supports fine-grained data security with column-level controls. Column-level security can be applied to local tables, views, and materialized views. Applying column-level security allows you to restrict access to personally identifiable information (PII) or payment card information (PCI) to selected people.

Column-level security enhances data compliance by restricting exposure of sensitive columns. Redshift Serverless provides flexibility to configure targeted column access control.

In this task, you implement secure access at the column-level by using GRANT and REVOKE to control permissions on columns for different users and roles.

21. **Command:** To connect to the Redshift endpoint as dbadmin user, run the following commands on the command host:

```
quit

psql -U dbadmin -h $RedshiftEndpoint -d $RedshiftDB -p 5439
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

psql (15.6, server 8.0.2)
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-
SHA384, bits: 256, compression: off)
Type "help" for help.

ticketdb=#
```

22. **Command:** To retrieve all the columns data from the customer table in the sales schema, run the following SELECT statement:

```
select * from sales.customer limit 10;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

 c_custkey |      c_name      |      c_address
| c_nationkey |      c_phone      | c_acctbal | c_mktsegment |
c_comment
```


23. **Command:** To revoke the SELECT permission from the sales_ro role, run the following command:

```
Revoke select on sales.customer from role sales_ro;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

REVOKE
```

You now set session authorization as data analyst user and validate (SELECT *) statement.

24. **Command:** To set temporary session authorization for jane_doe user, run the following command:

```
25. -- As Jane Doe - Data Analyst
26. SET SESSION AUTHORIZATION 'jane_doe';
    SELECT CURRENT_USER;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

SET
current_user
-----
jane_doe
(1 row)
```

25. **Command:** To retrieve all the columns data from the customer table in the sales schema, run the following SELECT statement:

```
select * from sales.customer limit 10;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

ERROR:  permission denied for relation customer
```

26. **Command:** To set authorization for dbadmin user, run the following commands on the command host:

```
27. SET SESSION AUTHORIZATION 'dbadmin';
    SELECT CURRENT_USER;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
```

```
*****
```

```
SET
  current_user
-----
dbadmin
(1 row)
```

27. **Command:** To grant the sales_ro role SELECT access only to the columns they need for analytical purposes, run the following command:
28. `GRANT SELECT (c_custkey, c_acctbal, c_mktsegment, c_nationkey, c_comment) ON sales.customer TO ROLE sales_ro;`

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
GRANT
```

You now set the session authorization as data analyst user and query the customer table by explicitly writing a SELECT statement that includes only the columns which have been granted access permission for the sales_ro role.

28. **Command:** To set temporary authorization for jane_doe user, run the following command:
29. `-- As Jane Doe - Data Analyst`
`SET SESSION AUTHORIZATION 'jane_doe';`

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
SET
```

29. **Command:** To select only the columns which have the grant permitting access for the sales_ro role, run the following SELECT statement:

```
select c_custkey, c_acctbal, c_mktsegment, c_nationkey, c_comment
from sales.customer limit 10;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
  c_custkey | c_acctbal | c_mktsegment | c_nationkey |
c_comment
-----+-----+-----+-----+-----
-----
-----
```

```

1 | 711.5600 | BUILDING | 15 | to the even,
regular platelets. regular, ironic epitaphs nag e
2 | 121.6500 | AUTOMOBILE | 13 | l accounts.
blithely ironic theodolites integrate boldly: caref
3 | 7498.1200 | AUTOMOBILE | 1 | deposits eat
slyly ironic, even instructions. express foxes detect slyly.
blithely even accounts abov
4 | 2866.8300 | MACHINERY | 4 | requests.
final, regular ideas sleep final accou
5 | 794.4700 | HOUSEHOLD | 3 | n accounts will
have to unwind. foxes cajole accor
6 | 7638.5700 | AUTOMOBILE | 20 | tions. even
deposits boost according to the slyly bold packages. final
accounts cajole requests. furious
7 | 9561.9500 | AUTOMOBILE | 18 | ainst the
ironic, express theodolites. express, even pinto beans among the
exp
8 | 6819.7400 | BUILDING | 17 | among the slyly
regular theodolites kindle blithely courts. carefully even
theodolites haggle slyly along the ide
9 | 8324.0700 | FURNITURE | 8 | r theodolites
according to the requests wake thinly excuses: pending requests
haggle furiousl
10 | 2753.5400 | HOUSEHOLD | 5 | es regular
deposits haggle. fur
(10 rows)

```

Consider: What happens if you try to use (SELECT *) or SELECT the columns that Data Analyst user Jane_Doe does not have permission to access?

Task complete: You have successfully implemented secure access at the column-level by using GRANT and REVOKE to control permissions on columns for different users and roles.

Task 4: Apply and manage Row-level security

With Row-level security, you can do the following:

- **Restrict row access based on roles:** The security admin creates and defines if a role can access specific records of data within a table based on an RLS policy.
- **Combine multiple policies per user or role:** Multiple policies can be defined per user or role, and all policies are applied with AND syntax.
- **Enhance granular access control:** RLS is built on role-based access control and can work alongside column-level access control.
- **No access if no policy applied:** All data access is blocked when there is no applicable policy on an RLS-protected table.

In this task, you apply row-level security (RLS) to restrict access to rows in a table based on the role and access policies. RLS filters sensitive data to ensure users only see permitted rows.

30. **Command:** To connect to the Redshift endpoint as dbadmin user, run the following commands:

```
quit
```

```
psql -U dbadmin -h $RedshiftEndpoint -d $RedshiftDB -p 5439
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
psql (15.6, server 8.0.2)  
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-  
SHA384, bits: 256, compression: off)  
Type "help" for help.
```

```
ticketdb=#
```

31. **Command:** To create the Data Engineer user `mary_major`, the Manager user `richard_roe`, their roles, and grant access to those roles, run the following statements:
32. `create user richard_roe with password 'AwsRSmgr@123';`
- 33.
34. `create user mary_major with password 'AwsRSde@123';`
- 35.
36. `create role sales_rw;`
- 37.
38. `grant usage on schema sales to role sales_rw;`
- 39.
40. `grant select, update, delete on all tables in schema sales to role sales_rw;`
- 41.
42. `grant role sales_rw to mary_major;`
- 43.
44. `create role select_all;`
- 45.
46. `grant usage on schema sales to role select_all;`
- 47.
48. `grant select on all tables in schema sales to role select_all;`
49. `grant role select_all to richard_roe;`

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
CREATE USER  
CREATE USER  
CREATE ROLE  
GRANT  
GRANT  
GRANT  
CREATE ROLE  
GRANT  
GRANT  
GRANT
```

32. **Command:** To check the data in customer table, run the following query:

Currently, data for each country is visible when querying those tables.

```
select c_nationkey,n_name,count(1) from sales.customer c join
sales.nation n
on c.c_nationkey=n.n_nationkey
group by 1,2 limit 10;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****

c_nationkey |      n_name      | count
-----+-----+-----
          18 | CHINA            | 4063
          20 | UNITED KINGDOM  | 3999
          10 | IRAN             | 3987
          19 | ROMANIA          | 4010
          17 | INDIA            | 3979
          16 | UNITED STATES    | 4032
           7 | GERMANY          | 3976
           4 | EGYPT            | 4014

(8 rows)
```

33. **Command:** To define a row-level security policy that allows the sales_ro role to only view data from their own country, run the following commands:

```
34. CREATE RLS POLICY data_analyst_policy
35. WITH (c_nationkey bigint) AS cs
36. USING (
37. cs.c_nationkey IN (select n_nationkey from sales.nation n join
sales.employee e on trim(n.n_name) = e.country_name and
employee_user_name = current_user and e.employee_status='A'));
38.
39. -- Grant select on Lookup tables
40. GRANT SELECT ON sales.nation ,sales.employee
41. TO RLS POLICY data_analyst_policy;
42.
43. -- Attach policy on the table to role
44. ATTACH RLS POLICY data_analyst_policy
45. ON sales.customer
46. TO ROLE sales_ro;
47.
48. --Enable RLS security on tables
ALTER TABLE sales.customer ROW LEVEL SECURITY on;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
CREATE RLS POLICY
GRANT
ATTACH RLS POLICY
ALTER TABLE
```

34. **Command:** To define a row level security policy for Data Engineers to restrict access based on region for the sales_rw role, run the following commands:

```
35. CREATE RLS POLICY data_engg_policy
36. WITH (c_nationkey bigint) AS cs
37. USING (cs.c_nationkey IN (
38. select n_nationkey from sales.nation n join sales.region r on
   n.n_regionkey=r.R_REGIONKEY join sales.employee e on
   trim(r.r_name)=e.region_name and employee_user_name =
   current_user and e.     employee_status='A'));
39.
40. -- Grant select on Lookup tables
41. GRANT SELECT ON sales.region, sales.nation ,sales.employee
42. TO RLS POLICY data_engg_policy;
43.
44. -- Attach policy on the table to role
45. ATTACH RLS POLICY data_engg_policy
46. ON sales.customer
   TO ROLE sales_rw;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****
```

```
CREATE RLS POLICY
GRANT
ATTACH RLS POLICY
```

35. **Command:** To define a policy that allows access to all rows for the Manager role, run the following commands:

```
36.
37. CREATE RLS POLICY mgr_select_all
38. USING ( true );
39.
40. ATTACH RLS POLICY mgr_select_all
41. ON sales.customer, sales.supplier
   TO ROLE select_all;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****
```

```
CREATE RLS POLICY
ATTACH RLS POLICY
```

36. **Command:** To test the row-level security policy created for Manager role, run the following commands:

Note: The manager role policy grants the manager user complete access to all data without any row-level filtering. This enables the manager to view all customer details across nations.

```
--- As Richard Roe - manager
```


37. **Command:** To test the row-level security policy created for Data Analyst role, run the following commands:

Note: The data analyst role policy grants user Jane Doe to perform country-level analysis, without overexposure of protected personal information.

```
-- As Jane Doe - Data Analyst

SET SESSION AUTHORIZATION 'jane_doe';

SELECT CURRENT_USER;

select c_nationkey,n_name,count(1) from sales.customer c join
sales.nation n
on c.c_nationkey = n. n_nationkey
group by 1,2;

Select * from sales.customer limit 10;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****

SET
  current_user
-----
  jane_doe
(1 row)

  c_nationkey |      n_name      | count
-----+-----+-----
          16 | UNITED STATES |    4032
(1 row)

  c_custkey | c_nationkey | c_acctbal | c_mktsegment |
c_comment
-----+-----+-----+-----+-----
-----
          44 |          16 | 7315.9400 | AUTOMOBILE   | r requests
around the unusual, bold a
          88 |          16 | 8031.4400 | AUTOMOBILE   | s are
quickly above the quickly ironic instructions; even requests
about the carefully final deposi
          90 |          16 | 7354.2300 | BUILDING     | sly across
the furiously even
         109 |          16 | -716.1000 | BUILDING     | es.
fluffily final dependencies sleep along the blithely even pinto
beans. final deposits haggle furiously furiou
         116 |          16 | 8403.9900 | BUILDING     | as. quickly
final sauternes haggle slyly carefully even packages. brave,
ironic pinto beans are above the furious
         137 |          16 | 7838.3000 | HOUSEHOLD    | carefully
regular theodolites use. silent dolphins cajo
```

```

143 |          16 | 2186.5000 | MACHINERY      | across the
blithely unusual requests haggle theodo
191 |          16 | 2945.1600 | BUILDING       | o beans
hinder slyly bold accounts.
194 |          16 | 6696.4900 | HOUSEHOLD      | quickly
across the fluffily dogged requests. regular platelets around the
ironic, even requests cajole quickl
200 |          16 | 9967.6000 | BUILDING       | e after the
ironic, even realms. fluffily regular packages doze-- courts
haggle carefully! blithely
(10 rows)

```

38. **Command:** To test the row-level security policy created for Data Engineer role, run the following commands:

Note: The data engineer role can only actually DELETE rows from their allowed region as per the policy restrictions. Trying to delete rows from regions they don't have access to will have no effect and delete zero rows after row-level security filtering is applied.

```

-- As Mary Major - Data Engineer

SET SESSION AUTHORIZATION 'mary_major';

SELECT CURRENT_USER;

select c_nationkey,n_name,count(1) from sales.customer c join
sales.nation n on c.c_nationkey=n.      n_nationkey
group by 1,2;

-- This user can delete data only from it's own Region (ASIA)
Delete from sales.customer USING sales.nation where
c_nationkey=n_nationkey
and n_name='INDIA' and c_acctbal < 0;

--Try to delete data from AMERICA (UNITED STATES)
Delete from sales.customer USING sales.nation where
c_nationkey=n_nationkey
and n_name='UNITED STATES' and c_acctbal < 0;

--- policies applied: data_engg_policy

```

Expected output:

```

*****
****  EXAMPLE OUTPUT  ****
*****

SET
  current_user
-----
  mary_major
(1 row)

  c_nationkey | n_name | count
-----+-----+-----
          17 | INDIA  |   3979
(1 row)

```

```
DELETE 378
DELETE 0
```

Task complete: You have successfully simplified the row-level security to achieve fine-grained access control on the confidential data.

Task 5: Configure dynamic data masking

Dynamic data masking makes it simple for you to adapt to changing privacy requirements without altering underlying data or updating SQL queries.

Amazon Redshift now extends security features by supporting Dynamic Data Masking (DDM) that allows you to simplify the process of protecting sensitive data in your Amazon Redshift data warehouse. With dynamic data masking, you control access to your data through simple SQL based masking policies that determine how Redshift returns sensitive data to the user at query time.

In this task, you define masking rules to hide sensitive columns based on user role.

This task describes four different data masking scenarios.

- Full data masking for regular users
- Partial data masking for analytics users
- Conditional data masking for fraud detection
- Full data access for auditors

39. **Command:** To create a sample test table that has sensitive data for credit card numbers, run the following commands:

```
40.
41. -- Create a sample test table that has sensitive data for credit
   card numbers.
42. SET SESSION AUTHORIZATION dbadmin;
43.
44. CREATE TABLE credit_cards (
45.     customer_id INT,
46.     fraud boolean,
47.     tran_date date,
48.     credit_card TEXT
49. );
50. );
51. INSERT INTO credit_cards
52. VALUES
53.     (100, 'Y' , current_date-10 , '4532993817514842'),
54.     (100, 'N', current_date-60, '4716002041425888'),
55.     (102, 'N', current_date-100, '5243112427642649'),
56.     (102, 'Y', current_date-90, '6011720771834675'),
57.     (102, 'N', current_date-120, '6011378662059710'),
58.     (103, 'Y', current_date-30, '373611968625635')
59. ;
60.
   select * from credit_cards;
```

Expected output:

```

*****
****  EXAMPLE OUTPUT  ****
*****

SET
CREATE TABLE
INSERT 0 6
  customer_id | fraud | tran_date | credit_card
-----+-----+-----+-----
          100 | t     | 2024-03-10 | 4532993817514842
          100 | f     | 2024-01-20 | 4716002041425888
          102 | f     | 2023-12-11 | 5243112427642649
          102 | t     | 2023-12-21 | 6011720771834675
          102 | f     | 2023-11-21 | 6011378662059710
          103 | t     | 2024-02-19 | 373611968625635
(6 rows)

```

Full data masking

40. **Command:** To mask full credit card data for regular users in Amazon Redshift Serverless, run the following commands:
- 41.
42. `SET SESSION AUTHORIZATION dbadmin;`
- 43.
44. `--Run GRANT to grant permission to use the SELECT statement on the table.`
45. `GRANT SELECT ON credit_cards TO PUBLIC;`
- 46.
47. `--create Regular user`
48. `CREATE USER regular_user WITH PASSWORD '1234Test!';`
- 49.
50. `--create a masking policy that fully masks the credit card number`
51. `CREATE MASKING POLICY mask_credit_card_full`
52. `WITH (credit_card VARCHAR(256))`
53. `USING ('000000XXXX0000'::TEXT);`
- 54.
55. `ATTACH MASKING POLICY mask_credit_card_full`
56. `ON credit_cards(credit_card)`
57. `TO PUBLIC;`
- 58.
- 59.
60. `SET SESSION AUTHORIZATION regular_user;`
- 61.
62. `SELECT * FROM credit_cards;`
`-- Credit card data are fully masked.`

Expected output:

```

*****
****  EXAMPLE OUTPUT  ****
*****

SET
GRANT
CREATE USER
CREATE MASKING POLICY

```

```
ATTACH MASKING POLICY
SET
```

customer_id	fraud	tran_date	credit_card
100	t	2024-03-10	000000XXXX0000
100	f	2024-01-20	000000XXXX0000
102	f	2023-12-11	000000XXXX0000
102	t	2023-12-21	000000XXXX0000
102	f	2023-11-21	000000XXXX0000
103	t	2024-02-19	000000XXXX0000

```
(6 rows)
```

Partial data masking

41. **Command:** To configure dynamic data masking to reveal the last 4 digits of credit card numbers for analytics users, run the following commands:

```
42.
43. SET SESSION AUTHORIZATION dbadmin;
44.
45. --create analytic user
46. CREATE USER analytics_user WITH PASSWORD '1234Test!';
47.
48. --create the analytics_role role and grant it to analytics_user
49. --regular_user does not have a role
50. CREATE ROLE analytics_role;
51.
52. GRANT ROLE analytics_role TO analytics_user;
53.
54. --create a user-defined function that partially obfuscates
    credit card data
55. CREATE FUNCTION REDACT_CREDIT_CARD (credit_card TEXT)
56. RETURNS TEXT IMMUTABLE
57. AS $$
58.     import re
59.     regexp = re.compile("^[0-9]{6}[0-9]{5,6}([0-9]{4})")
60.
61.     match = regexp.search(credit_card)
62.     if match != None:
63.         first = match.group(1)
64.         last = match.group(2)
65.     else:
66.         first = "000000"
67.         last = "0000"
68.
69.     return "{}XXXXX{}".format(first, last)
70. $$ LANGUAGE plpythonu;
71.
72. --create a masking policy that applies the REDACT_CREDIT_CARD
    function
73. CREATE MASKING POLICY mask_credit_card_partial
74. WITH (credit_card VARCHAR(256))
75. USING (REDACT_CREDIT_CARD(credit_card));
76.
77.
78. --attach mask_credit_card_partial to the analytics role
79. --users with the analytics role can see partial credit card
    information
```

```

80. ATTACH MASKING POLICY mask_credit_card_partial
81. ON credit_cards(credit_card)
82. TO ROLE analytics_role
83. PRIORITY 10;
84.
85. --confirm the partial masking policy is in place for users with
    the analytics role by selecting from      the credit card table as
    analytics_user
86. SET SESSION AUTHORIZATION analytics_user;
87.
    SELECT * FROM credit_cards;

```

Expected output:

```

*****
****  EXAMPLE OUTPUT  ****
*****

SET
CREATE USER
CREATE ROLE
GRANT
CREATE FUNCTION
CREATE MASKING POLICY
ATTACH MASKING POLICY
SET
customer_id | fraud | tran_date | credit_card
-----+-----+-----+-----
          100 | t     | 2024-03-10 | 453299XXXXX4842
          100 | f     | 2024-01-20 | 471600XXXXX5888
          102 | f     | 2023-12-11 | 524311XXXXX2649
          102 | t     | 2023-12-21 | 601172XXXXX4675
          102 | f     | 2023-11-21 | 601137XXXXX9710
          103 | t     | 2024-02-19 | 373611XXXXX5635
(6 rows)

```

Conditional data masking

42. **Command:** To configure dynamic data masking to reveal full credit card data only for the fraud team when the fraud flag is set to Yes, run the following commands:

```

43.
44. SET SESSION AUTHORIZATION dbadmin;
45.
46. CREATE USER riskmanagment_user WITH PASSWORD '1234Test!';
47.
48. --create the analytics_role role and grant it to analytics_user
49. --regular_user does not have a role
50. CREATE ROLE riskmanagment_role;
51.
52. GRANT ROLE riskmanagment_role TO riskmanagment_user;
53.
54. --Create a masking policy that partially redacts credit card
    numbers if the is_fraud value for that      row is TRUE,
55. --and otherwise blanks out the credit card number completely.
56.
57. CREATE MASKING POLICY mask_credit_card_conditional
58.     WITH (fraud BOOLEAN, pan varchar(256))

```

```

59.         USING (CASE WHEN fraud THEN REDACT_CREDIT_CARD(pan)
60.                 ELSE Null
61.             END);
62.
63. ATTACH MASKING POLICY mask_credit_card_conditional ON
    credit_cards (credit_card)
64.     USING (fraud, credit_card)
65.     TO ROLE riskmanagment_role PRIORITY 100;
66.
67. SET SESSION AUTHORIZATION riskmanagment_user;
68.
    SELECT * FROM credit_cards;

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

```

```

SET
CREATE USER
CREATE ROLE
GRANT
CREATE MASKING POLICY
ATTACH MASKING POLICY
SET

```

customer_id	fraud	tran_date	credit_card
100	t	2024-03-10	453299XXXXX4842
100	f	2024-01-20	
102	f	2023-12-11	
102	t	2023-12-21	601172XXXXX4675
102	f	2023-11-21	
103	t	2024-02-19	373611XXXXX5635

(6 rows)

Show entire credit card data to auditors

43. **Command:** To allow auditors to view unmasked credit card data in Amazon Redshift Serverless, run the following commands:

```

44.
45. SET SESSION AUTHORIZATION dbadmin;
46.
47. CREATE USER audit_user WITH PASSWORD '1234Test!';
48.
49. --create the analytics_role role and grant it to analytics_user
50. --regular_user does not have a role
51. CREATE ROLE audit_role;
52.
53. GRANT ROLE audit_role TO audit_user;
54.
55. --Create a masking policy that partially redacts credit card
    numbers if the is_fraud value for that row is TRUE,
56. --and otherwise blanks out the credit card number completely.
57. CREATE MASKING POLICY show_full_credit_card_number
58.     WITH (credit_card varchar(256))
59.     USING (credit_card);

```

```

60.
61. ATTACH MASKING POLICY show_full_credit_card_number ON
    credit_cards (credit_card)
62. USING (credit_card)
63. TO ROLE audit_role PRIORITY 15;
64.
65. SET SESSION AUTHORIZATION audit_user;
66.
    SELECT * FROM credit_cards;

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

SET
CREATE USER
CREATE ROLE
GRANT
CREATE MASKING POLICY
ATTACH MASKING POLICY
SET
  customer_id | fraud | tran_date | credit_card
-----+-----+-----+-----
          100 | t     | 2024-03-10 | 4532993817514842
          100 | f     | 2024-01-20 | 4716002041425888
          102 | f     | 2023-12-11 | 5243112427642649
          102 | t     | 2023-12-21 | 6011720771834675
          102 | f     | 2023-11-21 | 6011378662059710
          103 | t     | 2024-02-19 | 373611968625635
(6 rows)

```

Task complete: You have successfully implemented dynamic data masking in Redshift Serverless to securely hide confidential and regulated data from unauthorized access.

Task 6: Review Redshift Serverless audit logs

In this task, you launch Amazon CloudWatch Logs Insights and run a query to analyze Redshift Serverless audit logs.

44. At the top of the AWS Management Console, in the search bar, search for and choose

CloudWatch

45. In the left navigation pane, under **Logs**, choose **Logs Insights**.

46. From the **Select up to 50 log groups.**:
 - Select **/aws/redshift/labnamespace/useractivitylog**.
 - Select **/aws/redshift/labnamespace/connectionlog**.
 - Select **/aws/redshift/labnamespace/userlog**.

When you select a log group, CloudWatch Logs Insights automatically detects fields in the data in the log group.

47. **Command:** In the CloudWatch Logs Insights query editor window, paste the following query to view the latest 50 *labworkgrp* logs for different database users:

```
48. fields @timestamp, @message, @logStream
49. | filter @message like /dbadmin/ or @message like /jane_doe/ or
    @message like /mary_major/
    | sort @timestamp desc
```

48. Choose **Run query**.

This retrieves the filtered records into the query results based on the criteria given for analysis.

49. Choose the **1** icon to expand and observe the details.

Task complete: You have successfully ran a query to analyze Redshift Serverless user-activity, user, and connection logs.

Conclusion

You have successfully done the following:

- Created and managed users and roles.
- Applied and managed column-level security.
- Applied and managed row-level security.
- Configured dynamic data masking.
- Reviewed audit logs.

End lab